

S&Box

Module Requirements



Prepared By S&BOX
(Group 12)

1.Group members

- Ahmet Eren ARSLAN
- Beyzanur YILMAZ
- Mert UĞUR
- Mehmet Emin BAYTEKİN
- Bekir Sadık ALTUNKAYA
- Ahmet Hakan DEMİR
- Musab KARDEŞ
- Muhammet Talha MEMİŞOĞLU

2. Introduction

Game Name: S&BOX

Project Scope: This project aims to scan a physical surface containing sand and transform it into a playable game map. The variations in sand height will be analyzed to create a dynamic game environment that players can explore. If the shape of the terrain is altered, the game map will automatically update in real time to reflect the changes.

The Aim of The Project: The main goal of the project is to bridge the physical and digital worlds, providing players with a unique and interactive gaming experience. Players will be able to enjoy the game in a multiplayer mode.

3. Modules

Modules are divided to manage the project efficiently and distribute responsibilities across the team. The project consists of two main parts: game development and tool development. We divided it into 3 modules as follows:

- **Game Module:**

Covers the full development of the game, including core mechanics, real-time terrain updates based on Kinect data, multiplayer synchronization, and the user interface. It ensures smooth interaction and visual representation of the dynamic environment.

- **Kinect Module:**

Captures and processes depth data from the sand surface using Kinect, filters noise, and transfers the data to the game in real-time.

- **Server and Streaming Module:**

Hosts the multiplayer game via Unreal Engine Dedicated Server and streams it to web/mobile platforms using Pixel Streaming through Vagon.io.

Each module is designed to work together seamlessly, providing a real-time and interactive gaming experience across devices.

Kinect Module

Module Description:

This module constitutes the fundamental data collection and surface analysis component of the S&BOX project. It encompasses the processes of collecting depth data from physical surfaces using the Xbox Kinect One sensor, processing this data, and transferring it to the game environment in real-time. The module creates a bridge between the physical world and the digital game, ensuring that changes made by users on the sand surface are instantly reflected on the game map.

Requirements:

- **Data Collection and Processing:**
 - Continuous collection of depth data using the Xbox Kinect One sensor
 - Filtering of collected raw depth data and noise removal
 - Detection of height differences on the sand surface
 - Real-time data transfer to the server side when changes are detected
- **Calibration and Configuration:**
 - Positioning of the Kinect sensor at optimal distance and angle
 - Determination and limitation of the scanning area
 - Calibration settings for proper scaling of depth data
- **Integration:**
 - Configuration of the Kinect according to project requirements
 - Integration of processed data with the server module
 - Determination of protocol and format for real-time data transfer

Technologies and Tools:

- **Xbox Kinect One:** *Collection of depth image data*
- **C++:** Programming language used to process Kinect data
- **Data Processing Algorithms:** Depth map creation, filtering, and difference detection

Data Processing Flow:

- **Data Collection:** Continuous scanning of the sand surface using Xbox Kinect One and obtaining depth data
- **Pre-processing:** Filtering, noise removal, and normalization of raw depth data
- **Difference Analysis:** Detection of significant changes between current and previous scans
- **Data Transformation:** Conversion of depth data into a format usable by the game engine
- **Data Transfer:** Real-time transfer of processed data to the server module

Timeline and Milestones:

- Setup of Kinect sensor and configuration of basic data collection operations
- Development of depth data filtering and processing algorithms
- Implementation and optimization of the change detection system
- Completion of data integration with the server module
- Comprehensive testing and performance optimization

Test Cases:

Feature	Purpose	Test Input/Action	Expected Result	Result
Kinect Calibration	Verify Kinect sensor alignment and calibration accuracy	Power on the system and run standard Kinect alignment procedure	Kinect is aligned accurately and feedback is displayed on GUI	Planned
Lighting Condition Test	Evaluate sensor performance under different lighting environments	Test sensor performance in low light, normal room light, and bright light conditions	Sensor successfully detects depth and motion under all conditions	Planned
Data Transmission Test	Measure Kinect's data transfer speed and latency	Record transmission time of a Kinect depth frame from sensor to processing unit	Latency and bandwidth are within expected limits; results logged	Planned
Surface Change Detection	Validate detection accuracy for changes in surface height	Simulate gradual surface changes and observe Kinect response	Detection rate above 95%, logged with error margin analysis	Planned

Game Backend and Mechanics

Module Description:

This module covers the development of the game's core backend systems. It primarily focuses on processing and updating the dynamic 3D terrain captured through Kinect scans and gameplay. Additionally, it manages real-time terrain updates, player interactions, movement logic over changing surfaces, and collision detection. Multiplayer synchronization is also a critical aspect to ensure consistent terrain and gameplay states across all clients.

Requirements:

- **Terrain Integration:**
 - Convert Kinect-generated depth data into an in-game heightmap or mesh structure.
 - Continuously update the terrain in real-time as the sand formation changes.
- **Player Movement and Interaction:**
 - Develop movement logic that allows characters to navigate over dynamically changing terrain.
 - Define player interactions including physics-based behaviors and ability usage.
- **Gameplay Implementation:**
 - Design and implement core gameplay mechanics tailored to the dynamic terrain.
 - Finalize the game design elements such as objectives, challenges, and player roles.
- **Multiplayer Handling:**
 - Implement a networked game architecture to ensure synchronized terrain and gameplay across all clients.
 - Use server-authoritative logic to process Kinect data and distribute terrain updates to all players.
 - Manage player states, inputs, and interactions in a way that minimizes latency and maintains consistency.
 - Support session management features such as player joining, leaving, and reconnection handling.

Technologies and Tools:

- Unreal Engine / Custom C++ Engine: Used for rendering, physics simulation, and overall game development.
- Git: Employed for version control and collaborative development.

Timeline and Milestones:

- Terrain data parsing.

- Player movement system over dynamic terrain.
- Multiplayer architecture setup and synchronization testing.
- Development and refinement of game mechanics and rules.
- Comprehensive testing, debugging, and final optimization.

Test Cases:

Feature	Purpose	Test Input / Action	Expected Result	Result
Terrain Parsing Unit Test	Verify terrain data is parsed correctly for game environment	Run unit tests with various terrain files and check parsed output	Parsed terrain matches input data with no errors	Planned
Player Movement Unit Test	Ensure player movement logic functions correctly	Simulate user input and validate position updates in game loop	Player position updates accurately based on inputs	Planned
Multiplayer Sync Test	Confirm real-time synchronization between players	Simulate multiple clients performing actions simultaneously	All clients reflect consistent and synchronized game state	Planned
Core Gameplay Unit Test	Test key gameplay mechanics such as scoring and objectives	Trigger core events like score updates, win conditions, and objectives completion	Game logic executes correctly, and outcomes match expectations	Planned
Logging System	Track terrain and network event issues during runtime	Initiate terrain changes and multiplayer events with logging enabled	Logs show timestamps, event details, and any errors for debugging	Planned

Server & Streaming Module

Module Description

This module focuses on deploying and maintaining the infrastructure required for hosting the multiplayer Unreal Engine game via Dedicated Server and Pixel Streaming. The system will ensure accessibility across web and mobile platforms while preserving low-latency interactions and high-quality rendering. The cloud platform Vagon.io is used to manage the graphical workloads and deliver gameplay through browsers using WebRTC-based streaming.

Requirements

- **Multiplayer Server Hosting:**
 - Set up and deploy Unreal Engine Dedicated Server for multiplayer handling.
 - Ensure matchmaking, session joining, and player synchronization.
- **Cross-Platform Access:**
 - Allow gameplay access from web browsers and mobile devices
 - Use Pixel Streaming to stream the game from GPU cloud to user browsers.
- **Vagon.io Integration:**
 - Configure Vagon.io GPU machines to run the Pixel Streaming host.
 - Setup environment variables and signaling server

Technologies and Tools

- | | |
|----------------------|--|
| • Unreal Engine 4.27 | Game and server build, Pixel Streaming compatibility |
| • Pixel Streaming | WebRTC-based gameplay stream delivery |
| • Dedicated Server | Multiplayer logic, authority-based gameplay |
| • Vagon.io | Cloud GPU provider |
| • Node.js | For running signaling server |
| • Git | Version control & deployment pipeline |

Deployment Flow

1. Build Dedicated Server & Game Client.
2. Cook content and package project.
3. Upload to Vagon.io GPU instance.
4. Start Pixel Streaming signaling server.
5. Users connect via web/mobile through browser

Timeline and Milestones:

- Build and test Dedicated Server
- Package and cook game
- Integrate with Vagon.io
- Web & Mobile streaming

Test Cases:

Feature	Purpose	Test Input/Action	Expected Result	Result
Pixel Streaming Multiplayer	Verify multiplayer functionality over Pixel Streaming	Run server locally and connect multiple players	Game runs smoothly with multiple players connected	Done
Local Server Hardware Test	Check if our resources (ethernet, system adequacy, etc.) are sufficient for hosting.	Host the game with our own resources and monitor performance and latency	Local server runs game without performance issues	Planned
Server Deployment via Vagon.io	Ensure server can be accessed via cloud platform if our resources wouldn't be enough	Deploy game on Vagon.io and access via browser/mobile	Server is accessible from browser and mobile	Planned
WebRTC Signaling Connection	Test client-server WebRTC connection setup	Start signaling server and connect client	Clients can establish connection without errors	Planned
Real-time Terrain Update Stream	Confirm streamed game reflects live terrain changes	Modify terrain via Kinect and observe streamed output	Terrain changes are visible to all connected clients	Planned
Cross-Platform Access	Validate access and controls from different devices	Connect to game via desktop and mobile browsers	Game can be streamed and controlled on both platforms	Planned

Kinect	Game	Server
Musab KARDEŞ	Bekir Sadık ALTUNKAYA	Mert UĞUR
Beyzanur YILMAZ	Muhammet Talha MEMİŞOĞLU	Ahmet Hakan DEMİR
Ahmet Eren ARSLAN	Mehmet Emin BAYTEKİN	Beyzanur YILMAZ
Ahmet Hakan DEMİR	Ahmet Eren ARSLAN	Musab KARDEŞ
	Mert UĞUR	